Service-Level-Aware Integrated Scheduling with No-Regret Recoverability-Guided Rolling-Horizon Large Neighborhood Search

Siwen Liu[a,*]

[a]School of Information Science and Technology, Sandau University, Shanghai, 201209, PR China

ARTICLE INFO

Keywords:
Service-level scheduling
Flexible job shop scheduling
Integrated scheduling
Recoverability analysis
Rolling-horizon optimization
Large neighborhood search

ABSTRACT

This manuscript studies the Service-Level-Aware Integrated Scheduling Problem (SL-ISP), in which dynamically released flexible job-shop operations must be coordinated with aggregate service-level commitments of multiple service entities. The setting differs from classical tardiness-oriented shop scheduling because service feasibility depends on entity-level on-time quantity rather than only individual job due-date performance. A compact mixed-integer linear programming (MILP) model is provided as an offline small-instance benchmark and formal reference. To support online scheduling, a structural recoverability analysis is developed through optimistic service-recovery relaxation, recoverable quantity, unavoidable shortfall, recoverability erosion, and mandatory rescue jobs. These results motivate a solver-free No-Regret Recoverability-Guided Rolling-Horizon Large Neighborhood Search (NR-RG-RHO-LNS) algorithm that combines multi-start construction, recoverability-guided candidate diversity, rolling-horizon search, and incumbent preservation under the original objective. The computational section in this version defines the experimental protocol, compared algorithms, metrics, statistical tests, and all table and figure locations. Numerical findings and result-specific conclusions are intentionally left as placeholders until the experimental CSV files and generated figures are available.

1. Introduction

Instant fulfillment systems increasingly couple short-cycle production, order preparation, and service-level commitments. In such systems, a schedule is not evaluated solely by job completion times. A service entity may tolerate some late jobs if the committed on-time quantity is still met, while a small number of delayed high-quantity jobs may create an entity-level service violation. This feature creates a scheduling problem in which machine-level sequencing decisions and aggregate service-level fulfillment are inseparable.

This study considers the Service-Level-Aware Integrated Scheduling Problem (SL-ISP). Jobs arrive over a finite service window, each job belongs to one service entity, and each job consists of a sequence of flexible operations. After production completion, an entity-specific deterministic transportation delay is added before the job is counted against the entity cutoff time. Each entity has a required fulfillment ratio and a weight. The objective combines total tardiness (TT) and weighted service shortfall (WSF), and thus captures both continuous delivery delay and aggregate service-level violation.

The problem is related to integrated production and distribution scheduling, flexible job-shop scheduling, and dynamic rolling-horizon control. However, the service-level structure studied here is not an explicit vehicle-routing layer. Transportation delay is treated as a deterministic entity-level lead-time abstraction, while the main operational coupling occurs between shop-floor capacity and entity-level on-time quantity. This distinction is important: adding a routing layer would change the decision space, whereas SL-ISP focuses on how production schedules should anticipate service fulfillment thresholds.

The final algorithmic framework is No-Regret Recoverability-Guided Rolling-Horizon Large Neighborhood Search (NR-RG-RHO-LNS). It is designed as a solver-free heuristic for large dynamic instances, not as an exact optimizer. The main idea is to use lightweight recoverability indicators to produce service-risk-aware candidate diversity, while a no-regret selection rule prevents recoverability-guided candidates from being forced into the incumbent unless they improve the original objective Z.

siwenliu@sandau.edu.cn (S. Liu)
ORCID(s):

The main contributions are as follows.

1. A unified SL-ISP formulation is developed with flexible operations, entity-specific transportation delays, fulfillment-ratio commitments, and a composite tardiness-shortfall objective.
2. A compact MILP is given as an offline small-instance benchmark and formal reference model, with sequencing variables introduced only for operation pairs that can share a machine.
3. Structural recoverability results are derived, including optimistic recoverable quantity, unavoidable shortfall, recoverability erosion, and mandatory rescue jobs.
4. A solver-free NR-RG-RHO-LNS framework is proposed, combining no-regret multi-start initialization, recoverability-guided candidate generation, rolling-horizon large neighborhood search, and incumbent preservation.
5. A full computational protocol is specified with 13 compared algorithms, matched seeds, equal schedule-evaluation budgets for iterative methods, paired statistical tests, ablation analysis, scalability analysis, convergence curves, and a Gantt-chart illustration. Result-specific interpretations are deferred until the experimental outputs are generated.

2. Literature review

2.1. Integrated production and service-level scheduling

Integrated production and distribution scheduling has been studied across supply-chain, manufacturing, and time-window fulfillment contexts. Existing models often combine shop or flow-shop production with outbound delivery decisions, customer time windows, and earliness-tardiness penalties [8, 3]. Recent studies further consider distributed flexible job shops and transportation coordination, typically using evolutionary or memetic algorithms for large instances [10, 11, 6].

The SL-ISP differs from these settings in two respects. First, service feasibility is measured by an aggregate fulfillment ratio at the service-entity level, not by a customer-by-customer routing schedule. Second, the distribution component is represented by deterministic entity-level transportation delays, so the central decision is production sequencing under service-level commitments. This abstraction is appropriate when the downstream lead time is stable enough to be represented as an entity-level delay and the main operational bottleneck lies in production or order-preparation capacity.

2.2. Flexible job-shop and event-driven scheduling

Flexible job-shop scheduling extends classical job-shop scheduling by allowing each operation to choose among eligible machines. The resulting assignment-sequencing interaction substantially enlarges the search space. Recent work has explored learning-based and simulation-based approaches for flexible or dynamic shop scheduling, including deep reinforcement learning for flexible job shops [9], dynamic parallel-machine scheduling [7], and hierarchical multi-action policies for dynamic distributed job shops with arrivals [5].

Dynamic arrivals make event-driven rescheduling necessary. In SL-ISP, decision epochs are aligned with job releases, operation completions, and machine-idle events. This event-driven structure avoids solving a full static problem whenever the state changes, but it also requires the scheduler to diagnose whether the remaining service commitment is still recoverable. The recoverability analysis in Section 4 is introduced for this purpose.

2.3. Metaheuristic and rolling-horizon methods

Metaheuristics remain important for large scheduling problems because exact formulations become computationally expensive as sequencing decisions grow. Memetic algorithms, variable neighborhood search, tabu search, and hybrid evolutionary frameworks have been widely used in scheduling and integrated production-distribution problems [2, 1, 4]. Rolling-horizon optimization (RHO) decomposes a dynamic problem into shorter decision windows, while large neighborhood search (LNS) repairs selected portions of an incumbent schedule.

For SL-ISP, a direct exact rolling-horizon solver would be difficult to scale because each local subproblem inherits flexible machine assignment, precedence, non-preemption, and service-level shortfall coupling. The proposed algorithm therefore uses a solver-free RHO-LNS backbone. Recoverability-guided (RG) information is used to diversify construction and repair candidates, but all acceptance and final selection decisions are made using the original objective Z.

Table 1
Abbreviations.

Abbreviation	Meaning
SL-ISP	Service-Level-Aware Integrated Scheduling Problem
NR-RG-RHO-LNS	No-Regret Recoverability-Guided Rolling-Horizon Large Neighborhood Search
RHO	Rolling-horizon optimization
LNS	Large neighborhood search
RG	Recoverability-guided
EDF	Earliest deadline first
EDD	Earliest due date
SPT	Shortest processing time
WSPT	Weighted shortest processing time
ATC	Apparent tardiness cost
ILS	Iterated local search
VNS	Variable neighborhood search
TS	Tabu search
TT	Total tardiness
WSF	Weighted service shortfall
ZSR	Zero-shortfall entity rate
MILP	Mixed-integer linear programming

2.4. Research gaps and positioning of this study

Three gaps motivate this work. First, most integrated production-distribution models either include explicit routing decisions or use job-level due-date penalties; fewer formulations isolate aggregate service-level fulfillment as the main coupling structure. Second, dynamic flexible scheduling studies often optimize tardiness or makespan, while entity-level fulfillment ratios require a different diagnosis of residual risk. Third, metaheuristic rolling-horizon approaches usually rely on generic neighborhoods; they do not explicitly identify when a service entity is fragile, unrecoverable, or dependent on mandatory rescue jobs.

This study addresses these gaps by formulating SL-ISP, deriving structural recoverability indicators, and embedding those indicators in a no-regret RHO-LNS algorithm. The resulting method is positioned as a scalable heuristic framework for service-level-aware integrated scheduling, with the MILP used only for formal reference and small-instance benchmarking.

3. Problem description and mathematical formulation

3.1. Problem description

Before presenting the model, Table 1 lists the abbreviations used throughout the manuscript and Table 2 summarizes the main notation. Algorithmic operator names and experiment-only fields are intentionally excluded from the notation table.

SL-ISP is defined over a finite service window. A set of jobs $\mathcal{J}$ must be processed on a set of flexible machines $\mathcal{M}$ while satisfying service commitments for a set of service entities $\mathcal{R}$. Each job $j \in \mathcal{J}$ is assigned to exactly one service entity through the mapping $g(j) \in \mathcal{R}$.

Job j has a release time r_j, a quantity contribution q_j, and an ordered operation sequence $\mathcal{O}_j = \{1, \dots, n_j\}$. Operation O_{jo} can be processed on one eligible machine $h \in \mathcal{M}_{jo}$, and its processing time on that machine is p_{joh}. Operations of the same job must follow their given precedence order, and each operation is non-preemptive. A machine can process at most one operation at a time.

Each service entity $r \in \mathcal{R}$ has a service cutoff time d_r, deterministic transportation delay τ_r, required fulfillment ratio ρ_r, and weight w_r. The delay τ_r is an entity-level lead-time abstraction and does not represent explicit vehicle routing or batching. The aggregate quantity and minimum required on-time quantity are

$$Q_r = \sum_{j\,:\,g(j)=r} q_j, \qquad Q_r^{\min} = \rho_r Q_r. \tag{1}$$

Table 2
Notation for SL-ISP and recoverability analysis.

Symbol	Definition
Sets	
$\mathcal{J}$	Set of jobs.
$\mathcal{M}$	Set of machines.
$\mathcal{R}$	Set of service entities.
$\mathcal{O}_j$	Ordered set of operations of job j.
$\mathcal{M}_{jo}$	Eligible machines of operation O_{jo}.
$\mathcal{A}$	Set of all operations, $\mathcal{A} = \{(j,o) : j \in \mathcal{J}, o \in \mathcal{O}_j\}$.
$\mathcal{P}_h$	Operation pairs that can compete for machine h.
Parameters	
r_j	Release time of job j.
p_{joh}	Processing time of operation O_{jo} on machine h.
q_j	Quantity contribution of job j.
$g(j)$	Service-entity mapping of job j.
d_r	Service cutoff time of entity r.
τ_r	Deterministic transportation delay of entity r.
ρ_r	Required fulfillment ratio of entity r.
w_r	Weight of entity r.
Q_r	Total quantity of entity r, $Q_r = \sum_{j:g(j)=r} q_j$.
$Q_r^{\min}$	Minimum required on-time quantity, $Q_r^{\min} = \rho_r Q_r$.
α, β	Objective weights for tardiness and weighted shortfall.
H	Scheduling horizon and Big-M constant.
Decision variables and derived schedule quantities	
S_{jo}	Start time of operation O_{jo}.
C_j	Production completion time of job j.
D_j	Delivery time of job j.
T_j	Tardiness of job j.
z_j	On-time indicator of job j.
Q_r^{on}	On-time quantity delivered for entity r.
U_r	Entity-level service shortfall.
X_{joh}	Binary assignment variable equal to 1 if O_{jo} is assigned to machine h.
$Y_{jo,iqh}$	Binary sequencing variable for two operations on machine h.
Recoverability terms	
t	Current decision time.
$Q_r^{sec}(t)$	Secured on-time quantity of entity r at time t.
$Q_r^{rem}(t)$	Remaining required quantity of entity r at time t.
$\underline{D}_j(t)$	Optimistic lower bound on delivery time of job j from time t.
$\mathcal{E}_r(t)$	Jobs of entity r that can optimistically meet d_r.
$\mathcal{B}$	Bottleneck resource pool used in the recoverability relaxation.
$\mathrm{Cap}_{\mathcal{B}}(t, d_r)$	Available capacity of $\mathcal{B}$ before d_r.
$Q_{r,\mathcal{B}}^{rec}(t)$	Maximum recoverable on-time quantity under the optimistic relaxation.
$U_{r,\mathcal{B}}(t)$	Unavoidable shortfall lower bound under the optimistic relaxation.
$S_{r,\mathcal{B}}^{rec}(t)$	Recoverability surplus, $Q_{r,\mathcal{B}}^{rec}(t) - Q_r^{rem}(t)$.
$\Delta_{j,r,\mathcal{B}}^{rec}(t)$	Marginal recoverability contribution of job j.

Let C_j be the production completion time of the final operation of job j. The delivery time is

$$D_j = C_j + \tau_{g(j)}. \tag{2}$$

Job j is on time if $D_j \le d_{g(j)}$, which is represented by z_j. Its tardiness is

$$T_j = [D_j - d_{g(j)}]_+, \tag{3}$$

where $[x]_+ = \max\{0, x\}$. The on-time quantity and service shortfall of entity r are

$$Q_r^{on} = \sum_{j:g(j)=r} q_j z_j, \tag{4}$$

$$U_r = [Q_r^{\min} - Q_r^{on}]_+. \tag{5}$$

The system is rescheduled in an event-driven manner. Decision epochs are aligned with job-release events, operation-completion events, and machine-idle events. At each epoch, the schedule state is updated and feasible ready operations may be assigned to idle machines. The objective is

$$\min Z = \alpha \sum_{j\in\mathcal{J}} T_j + \beta \sum_{r\in\mathcal{R}} w_r U_r. \tag{6}$$

The ratio β/α represents the relative objective emphasis on service shortfall versus tardiness; it is not treated as a separate service-pressure parameter in the model.

3.2. Mathematical programming model

This subsection provides a MILP formulation for a fully revealed service window. The formulation is an offline benchmark and formal reference model. It is not the large-scale solution method used in the computational study; the proposed NR-RG-RHO-LNS algorithm in Section 5 is solver-free.

Define the operation set

$$\mathcal{A} = \{(j, o) : j \in \mathcal{J},\ o \in \mathcal{O}_j\}, \tag{7}$$

and, for each machine $h \in \mathcal{M}$, define the pair set

$$\mathcal{P}_h = \{((j, o), (i, q)) : (j, o) <_{\text{lex}} (i, q),\ h \in \mathcal{M}_{jo} \cap \mathcal{M}_{iq}\}. \tag{8}$$

The lexicographic order is arbitrary and is used only to avoid duplicate operation pairs. Sequencing variables are introduced only for $\mathcal{P}_h$, i.e., only when two operations can compete for the same machine.

The offline MILP is

$$\min Z = \alpha \sum_{j\in\mathcal{J}} T_j + \beta \sum_{r\in\mathcal{R}} w_r U_r \tag{9}$$

subject to

$$\sum_{h\in\mathcal{M}_{jo}} X_{joh} = 1, \qquad \forall j \in \mathcal{J},\ o \in \mathcal{O}_j, \tag{10}$$

$$S_{j1} \geq r_j, \qquad \forall j \in \mathcal{J}, \tag{11}$$

$$S_{j,o+1} \geq S_{jo} + \sum_{h\in\mathcal{M}_{jo}} p_{joh} X_{joh}, \qquad \forall j \in \mathcal{J},\ o = 1, \ldots, n_j - 1, \tag{12}$$

$$S_{jo} + p_{joh} \leq S_{iq} + H(3 - X_{joh} - X_{iqh} - Y_{jo,iqh}), \qquad \forall h \in \mathcal{M},\ ((j, o), (i, q)) \in \mathcal{P}_h, \tag{13}$$

$$S_{iq} + p_{iqh} \leq S_{jo} + H(2 - X_{joh} - X_{iqh} + Y_{jo,iqh}), \qquad \forall h \in \mathcal{M},\ ((j, o), (i, q)) \in \mathcal{P}_h, \tag{14}$$

$$C_j = S_{j,n_j} + \sum_{h\in\mathcal{M}_{j,n_j}} p_{j,n_j,h} X_{j,n_j,h}, \qquad \forall j \in \mathcal{J}, \tag{15}$$

$$D_j = C_j + \tau_{g(j)}, \qquad \forall j \in \mathcal{J}, \tag{16}$$

$$T_j \geq D_j - d_{g(j)}, \qquad \forall j \in \mathcal{J}, \tag{17}$$

$$D_j \leq d_{g(j)} + H(1 - z_j), \qquad \forall j \in \mathcal{J}, \tag{18}$$

$$U_r \geq \rho_r Q_r - \sum_{j:g(j)=r} q_j z_j, \qquad \forall r \in \mathcal{R}, \tag{19}$$

$$X_{joh} \in \{0, 1\}, \qquad \forall j \in \mathcal{J},\ o \in \mathcal{O}_j,\ h \in \mathcal{M}_{jo}, \tag{20}$$

$$Y_{jo,iqh} \in \{0,1\}, \qquad \forall h \in \mathcal{M},\ ((j,o),(i,q)) \in \mathcal{P}_h, \tag{21}$$

$$z_j \in \{0,1\}, \qquad \forall j \in \mathcal{J}, \tag{22}$$

$$S_{jo}, C_j, D_j, T_j \geq 0, \qquad \forall j \in \mathcal{J},\ o \in \mathcal{O}_j, \tag{23}$$

$$U_r \geq 0, \qquad \forall r \in \mathcal{R}. \tag{24}$$

Constraints (10)–(12) model machine assignment, release times, and within-job precedence. Constraints (13)–(14) enforce non-overlap only for operation pairs in $\mathcal{P}_h$. Constraints (15)–(17) define completion, delivery, and tardiness. Constraint (18) ensures that $z_j = 1$ is possible only if job j is delivered by the service cutoff of its entity. Because z_j can reduce the shortfall term in (19), objective (9) pushes feasible on-time indicators to one whenever doing so reduces U_r; remaining ties can be resolved without changing Z. Constraints (20)–(24) give the complete variable domains.

The number of pairwise sequencing variables grows with the number of machine-compatible operation pairs. Consequently, the MILP is suitable mainly for small offline benchmarks and model validation. Large dynamic instances are handled by the solver-free NR-RG-RHO-LNS algorithm.

3.3. Complexity and motivation for heuristic solution

A central structural question is whether zero shortfall can be attained for a given instance. Even a highly restricted version of SL-ISP is computationally intractable.

Definition 1 (Restricted zero-shortfall service fulfillment problem). An instance of the restricted zero-shortfall service fulfillment problem consists of one service entity, one machine, single-operation jobs released at time zero, zero transportation delay, and a common cutoff d. Each job $j \in \mathcal{J}$ has processing time p_j and quantity q_j. Given a required on-time quantity Q, the question is whether there exists a subset of jobs whose total processing time is at most d and whose total quantity is at least Q.

Theorem 1 (Complexity of zero-shortfall verification). *The restricted zero-shortfall service fulfillment problem is NP-complete.*

PROOF. Membership in NP is immediate. A certificate is a subset $S \subseteq \mathcal{J}$. Verifying $\sum_{j\in S} p_j \leq d$ and $\sum_{j\in S} q_j \geq Q$ requires $O(|\mathcal{J}|)$ time.

For NP-hardness, reduce from the 0-1 knapsack decision problem. Given items with weights a_i, values b_i, capacity A, and target value B, construct one job for each item with processing time $p_j = a_i$ and quantity $q_j = b_i$. Set the common cutoff to $d = A$, set the required on-time quantity to $Q = B$, use one machine, set all release times to zero, and set the transportation delay to zero. In this single-machine setting, a selected set of jobs completes by the cutoff if and only if the corresponding item set has total weight at most A, and its delivered quantity reaches the service requirement if and only if the corresponding item set has value at least B. Thus the knapsack instance is feasible if and only if the constructed scheduling instance admits zero shortfall. The transformation is polynomial, so the restricted problem is NP-hard. Together with membership in NP, it is NP-complete.

Theorem 1 justifies heuristic solution for large SL-ISP instances and motivates recoverability indicators that are cheaper than exact zero-shortfall verification.

4. Structural recoverability analysis

4.1. Optimistic recoverability relaxation

To support event-driven diagnosis, we use an optimistic relaxation that ignores future sequencing interference except for capacity consumed by a designated bottleneck resource pool $\mathcal{B} \subseteq \mathcal{M}$. In the structural results below, $\mathcal{B}$ is interpreted as an unavoidable capacity pool for the selected recovery workload, so the workload term defined in (29) is a valid lower bound on the pool capacity required by a selected job subset. At decision time t, the secured on-time quantity of entity r is

$$Q_r^{sec}(t) = \sum_{\substack{j:g(j)=r \\ C_j \leq t,\ D_j \leq d_r}} q_j, \tag{25}$$

and the remaining required quantity is

$$Q_r^{rem}(t) = [Q_r^{\min} - Q_r^{sec}(t)]_+. \tag{26}$$

For an unfinished job j, let $\mathcal{O}_j^{rem}(t)$ denote its remaining operations, with residual processing used for an ongoing non-preemptive operation. An optimistic delivery-time lower bound is

$$\underline{D}_j(t) = \max\{t, r_j\} + \sum_{o \in \mathcal{O}_j^{rem}(t)} \min_{h \in \mathcal{M}_{jo}} p_{joh} + \tau_{g(j)}. \tag{27}$$

This bound assumes zero waiting and the fastest eligible machine for each remaining operation. The eligible recovery set is

$$\mathcal{E}_r(t) = \{j \in \mathcal{J} : g(j) = r,\ j \text{ not secured at } t,\ \underline{D}_j(t) \le d_r\}. \tag{28}$$

Let $\mathrm{Cap}_{\mathcal{B}}(t, d_r)$ be the aggregate available capacity of $\mathcal{B}$ during $[t, d_r]$, net of already committed non-preemptive work. For job j, define its minimum bottleneck workload

$$\underline{w}_{j,\mathcal{B}}(t) = \sum_{o \in \mathcal{O}_j^{rem}(t)} \min_{h \in \mathcal{B} \cap \mathcal{M}_{jo}} p_{joh}, \tag{29}$$

with the convention that $\min_{h \in \emptyset} p_{joh} = 0$. This term is an optimistic lower bound on the bottleneck-pool workload counted for job j in the relaxation.

The minimum-workload recovery problem asks how much bottleneck workload is needed to assemble the remaining required quantity:

$$W_{r,\mathcal{B}}^{\min}(t) = \min \sum_{j \in \mathcal{E}_r(t)} \underline{w}_{j,\mathcal{B}}(t) y_j \tag{30}$$

subject to

$$\sum_{j \in \mathcal{E}_r(t)} q_j y_j \ge Q_r^{rem}(t), \qquad y_j \in \{0, 1\}. \tag{31}$$

If no subset of $\mathcal{E}_r(t)$ meets the quantity requirement, set $W_{r,\mathcal{B}}^{\min}(t) = +\infty$.

The maximum recoverable quantity problem is

$$Q_{r,\mathcal{B}}^{rec}(t) = \max \sum_{j \in \mathcal{E}_r(t)} q_j x_j \tag{32}$$

subject to

$$\sum_{j \in \mathcal{E}_r(t)} \underline{w}_{j,\mathcal{B}}(t) x_j \le \mathrm{Cap}_{\mathcal{B}}(t, d_r), \qquad x_j \in \{0, 1\}. \tag{33}$$

The selectors x_j and y_j are local variables of the relaxation. The two problems form a dual pair of 0-1 knapsack views: one minimizes workload for a quantity target, and the other maximizes quantity under a workload budget.

4.2. Recoverable quantity and unavoidable shortfall

Theorem 2 (Equivalent characterizations of optimistic zero-shortfall recoverability). *For any entity $r \in \mathcal{R}$ at decision time t,*

$$W_{r,\mathcal{B}}^{\min}(t) \le \mathrm{Cap}_{\mathcal{B}}(t, d_r) \quad \Longleftrightarrow \quad Q_{r,\mathcal{B}}^{rec}(t) \ge Q_r^{rem}(t). \tag{34}$$

PROOF. First assume $W_{r,\mathcal{B}}^{\min}(t) \le \mathrm{Cap}_{\mathcal{B}}(t, d_r)$. Let y^* be an optimal solution of (30)–(31). Then

$$\sum_{j \in \mathcal{E}_r(t)} \underline{w}_{j,\mathcal{B}}(t) y_j^* = W_{r,\mathcal{B}}^{\min}(t) \le \mathrm{Cap}_{\mathcal{B}}(t, d_r),$$

and $\sum_{j\in\mathcal{E}_r(t)} q_j y_j^* \geq Q_r^{rem}(t)$. Setting $x_j = y_j^*$ gives a feasible solution to (32)–(33) with value at least $Q_r^{rem}(t)$. Hence $Q_{r,\mathcal{B}}^{rec}(t) \geq Q_r^{rem}(t)$.

Conversely, assume $Q_{r,\mathcal{B}}^{rec}(t) \geq Q_r^{rem}(t)$. Let x^* be an optimal solution of (32)–(33). It satisfies

$$\sum_{j\in\mathcal{E}_r(t)} q_j x_j^* = Q_{r,\mathcal{B}}^{rec}(t) \geq Q_r^{rem}(t)$$

and

$$\sum_{j\in\mathcal{E}_r(t)} \underline{w}_{j,\mathcal{B}}(t) x_j^* \leq \mathrm{Cap}_{\mathcal{B}}(t, d_r).$$

Setting $y_j = x_j^*$ gives a feasible solution to (30)–(31). Therefore

$$W_{r,\mathcal{B}}^{\min}(t) \leq \sum_{j\in\mathcal{E}_r(t)} \underline{w}_{j,\mathcal{B}}(t) x_j^* \leq \mathrm{Cap}_{\mathcal{B}}(t, d_r).$$

Both directions hold.

Implication for NR-RG-RHO-LNS: Theorem 2 provides two equivalent online diagnoses, enabling either a workload-fit view or a recoverable-quantity view when generating RG candidates.

The unavoidable shortfall lower bound induced by the optimistic relaxation is

$$U_{r,\mathcal{B}}(t) = [Q_r^{rem}(t) - Q_{r,\mathcal{B}}^{rec}(t)]_+. \tag{35}$$

Theorem 3 (Unavoidable shortfall lower bound). *For any feasible continuation schedule π from time t, the realized shortfall of entity r satisfies*

$$U_r^{\pi} \geq U_{r,\mathcal{B}}(t). \tag{36}$$

Proof. Let $\mathcal{J}_r^{on}(\pi) \subseteq \mathcal{E}_r(t)$ be the set of entity-r jobs that are not secured at time t but are delivered on time under continuation π. Every such job must complete its remaining operations, and the total bottleneck workload assigned to $\mathcal{B}$ before d_r cannot exceed $\mathrm{Cap}_{\mathcal{B}}(t, d_r)$. Therefore,

$$\sum_{j\in\mathcal{J}_r^{on}(\pi)} \underline{w}_{j,\mathcal{B}}(t) \leq \mathrm{Cap}_{\mathcal{B}}(t, d_r).$$

The indicator vector of $\mathcal{J}_r^{on}(\pi)$ is feasible for (32)–(33); hence

$$\sum_{j\in\mathcal{J}_r^{on}(\pi)} q_j \leq Q_{r,\mathcal{B}}^{rec}(t).$$

The total on-time quantity under π is $Q_r^{sec}(t) + \sum_{j\in\mathcal{J}_r^{on}(\pi)} q_j$, so

$$\begin{aligned} U_r^{\pi} &= \left[Q_r^{\min} - Q_r^{sec}(t) - \sum_{j\in\mathcal{J}_r^{on}(\pi)} q_j \right]_+ \\ &= \left[Q_r^{rem}(t) - \sum_{j\in\mathcal{J}_r^{on}(\pi)} q_j \right]_+ \\ &\geq [Q_r^{rem}(t) - Q_{r,\mathcal{B}}^{rec}(t)]_+ = U_{r,\mathcal{B}}(t), \end{aligned}$$

where the inequality follows from monotonicity of $[\cdot]_+$.

Implication for NR-RG-RHO-LNS: $U_{r,\mathcal{B}}(t)$ gives a certificate of residual service risk that is valid for all feasible continuations of the current schedule state.

Corollary 1 (Unattainability of zero shortfall). *If $Q^{rec}_{r,\mathcal{B}}(t) < Q^{rem}_{r}(t)$, then $U^{\pi}_{r} > 0$ for every feasible continuation schedule π.*

PROOF. The condition $Q^{rec}_{r,\mathcal{B}}(t) < Q^{rem}_{r}(t)$ implies $U_{r,\mathcal{B}}(t) > 0$. The claim follows directly from Theorem 3.

Implication for NR-RG-RHO-LNS: the algorithm can avoid treating zero shortfall for such an entity as an enforceable target, while still using recoverability information to construct service-risk-aware candidate diversity.

4.3. Recoverability erosion

Recoverability is time-sensitive because capacity consumption and shrinking slack can reduce the set of recoverable jobs. The following result restores the monotonic erosion statement under the same conditions used in the original structural analysis.

Theorem 4 (Recoverability erosion under non-recovery capacity consumption). *Let $t_1 < t_2$ be two decision epochs. Suppose that, for entity r,*

(C1) $Q^{rem}_{r}(t_2) = Q^{rem}_{r}(t_1)$;
(C2) $\mathcal{E}_r(t_2) \subseteq \mathcal{E}_r(t_1)$;
(C3) $\underline{w}_{j,\mathcal{B}}(t_2) \geq \underline{w}_{j,\mathcal{B}}(t_1)$ *for all* $j \in \mathcal{E}_r(t_2)$;
(C4) $\mathrm{Cap}_{\mathcal{B}}(t_2, d_r) \leq \mathrm{Cap}_{\mathcal{B}}(t_1, d_r)$.

Then

$$Q^{rec}_{r,\mathcal{B}}(t_2) \leq Q^{rec}_{r,\mathcal{B}}(t_1), \tag{37}$$

$$U_{r,\mathcal{B}}(t_2) \geq U_{r,\mathcal{B}}(t_1). \tag{38}$$

PROOF. Let x^* be an optimal solution of the maximum recoverable quantity problem at t_2, so that

$$Q^{rec}_{r,\mathcal{B}}(t_2) = \sum_{j \in \mathcal{E}_r(t_2)} q_j x^*_j.$$

By (C2), every job selected by x^* belongs to $\mathcal{E}_r(t_1)$. Construct a candidate solution $\hat{x}$ for time t_1 by setting $\hat{x}_j = x^*_j$ for $j \in \mathcal{E}_r(t_2)$ and $\hat{x}_j = 0$ for $j \in \mathcal{E}_r(t_1) \setminus \mathcal{E}_r(t_2)$. Its bottleneck workload at t_1 satisfies

$$\begin{aligned}
\sum_{j \in \mathcal{E}_r(t_1)} \underline{w}_{j,\mathcal{B}}(t_1)\hat{x}_j &= \sum_{j \in \mathcal{E}_r(t_2)} \underline{w}_{j,\mathcal{B}}(t_1)x^*_j \\
&\leq \sum_{j \in \mathcal{E}_r(t_2)} \underline{w}_{j,\mathcal{B}}(t_2)x^*_j \qquad \text{by (C3)} \\
&\leq \mathrm{Cap}_{\mathcal{B}}(t_2, d_r) \qquad \text{by feasibility at } t_2 \\
&\leq \mathrm{Cap}_{\mathcal{B}}(t_1, d_r) \qquad \text{by (C4).}
\end{aligned}$$

Thus $\hat{x}$ is feasible at t_1, and

$$Q^{rec}_{r,\mathcal{B}}(t_1) \geq \sum_{j \in \mathcal{E}_r(t_1)} q_j \hat{x}_j = \sum_{j \in \mathcal{E}_r(t_2)} q_j x^*_j = Q^{rec}_{r,\mathcal{B}}(t_2),$$

which proves (37). Using (C1) and (37),

$$\begin{aligned}
U_{r,\mathcal{B}}(t_2) &= [Q^{rem}_{r}(t_2) - Q^{rec}_{r,\mathcal{B}}(t_2)]_+ \\
&\geq [Q^{rem}_{r}(t_1) - Q^{rec}_{r,\mathcal{B}}(t_1)]_+ = U_{r,\mathcal{B}}(t_1),
\end{aligned}$$

which proves (38).

Implication for NR-RG-RHO-LNS: erosion identifies fragile entities whose recoverability margin is being consumed, motivating guarded RG intensification before service recovery becomes structurally impossible. The result does not claim that recoverability always decreases; new arrivals or capacity releases may violate the stated conditions.

4.4. Mandatory rescue jobs

When optimistic zero-shortfall recovery remains attainable, some jobs may be indispensable. For $j \in \mathcal{E}_r(t)$, define the exclusion-based maximum recoverable quantity as

$$Q_{r,\mathcal{B}}^{rec,-j}(t) = \max \sum_{i \in \mathcal{E}_r(t) \setminus \{j\}} q_i x_i \tag{39}$$

subject to

$$\sum_{i \in \mathcal{E}_r(t) \setminus \{j\}} \underline{w}_{i,\mathcal{B}}(t) x_i \le \mathrm{Cap}_{\mathcal{B}}(t, d_r), \qquad x_i \in \{0,1\}. \tag{40}$$

Definition 2 (Mandatory rescue job). A job $j \in \mathcal{E}_r(t)$ is a mandatory rescue job for entity r at time t if

$$Q_{r,\mathcal{B}}^{rec}(t) \ge Q_r^{rem}(t) \quad \text{and} \quad Q_{r,\mathcal{B}}^{rec,-j}(t) < Q_r^{rem}(t). \tag{41}$$

Theorem 5 (Mandatory on-time delivery). *If j is a mandatory rescue job for entity r at time t, then every feasible continuation schedule π with $U_r^\pi = 0$ must deliver j no later than d_r.*

PROOF. Suppose, for contradiction, that a feasible continuation π achieves $U_r^\pi = 0$ while delivering j after the cutoff, i.e., $D_j^\pi > d_r$. Let $\mathcal{J}_r^{on}(\pi) \subseteq \mathcal{E}_r(t)$ be the entity-r jobs delivered on time under π. Since j is late, $\mathcal{J}_r^{on}(\pi) \subseteq \mathcal{E}_r(t) \setminus \{j\}$. Because $U_r^\pi = 0$, the selected on-time jobs must satisfy

$$\sum_{i \in \mathcal{J}_r^{on}(\pi)} q_i \ge Q_r^{rem}(t).$$

As in the proof of Theorem 3, feasibility of π implies

$$\sum_{i \in \mathcal{J}_r^{on}(\pi)} \underline{w}_{i,\mathcal{B}}(t) \le \mathrm{Cap}_{\mathcal{B}}(t, d_r).$$

Thus the indicator vector of $\mathcal{J}_r^{on}(\pi)$ is feasible for the exclusion problem (39)–(40), giving

$$Q_{r,\mathcal{B}}^{rec,-j}(t) \ge \sum_{i \in \mathcal{J}_r^{on}(\pi)} q_i \ge Q_r^{rem}(t),$$

which contradicts Definition 2. Therefore any zero-shortfall continuation must deliver j on time.

Implication for NR-RG-RHO-LNS: mandatory rescue jobs are direct candidates for RG repair, priority construction, and no-regret candidate filtering.

Proposition 1 (Quantity-mandatory jobs). *Assume $Q_{r,\mathcal{B}}^{rec}(t) \ge Q_r^{rem}(t)$. If*

$$\sum_{i \in \mathcal{E}_r(t) \setminus \{j\}} q_i < Q_r^{rem}(t), \tag{42}$$

then j is a mandatory rescue job.

PROOF. Any feasible solution of the exclusion problem (39)–(40) selects only jobs from $\mathcal{E}_r(t) \setminus \{j\}$. Hence its value cannot exceed the total available quantity in that set:

$$Q_{r,\mathcal{B}}^{rec,-j}(t) \le \sum_{i \in \mathcal{E}_r(t) \setminus \{j\}} q_i < Q_r^{rem}(t).$$

Together with $Q_{r,\mathcal{B}}^{rec}(t) \ge Q_r^{rem}(t)$, Definition 2 is satisfied.

Implication for NR-RG-RHO-LNS: quantity-mandatory jobs can be detected without solving exclusion knapsack problems, making them inexpensive rescue candidates.

Proposition 2 (Capacity-mandatory jobs). *Assume $Q^{rec}_{r,\mathcal{B}}(t) \geq Q^{rem}_{r}(t)$. If*

$$\sum_{i \in \mathcal{E}_r(t) \setminus \{j\}} q_i \geq Q^{rem}_{r}(t) \quad but \quad Q^{rec,-j}_{r,\mathcal{B}}(t) < Q^{rem}_{r}(t), \tag{43}$$

then j is a mandatory rescue job.

PROOF. The first inequality rules out pure quantity insufficiency after excluding j, while the second states that no capacity-feasible alternative subset can meet the remaining requirement. With $Q^{rec}_{r,\mathcal{B}}(t) \geq Q^{rem}_{r}(t)$, these conditions are exactly Definition 2.

Implication for NR-RG-RHO-LNS: capacity-mandatory jobs expose service risk caused by bottleneck contention rather than aggregate quantity shortage.

Proposition 3 (Marginal recoverability characterization). *Let*

$$S^{rec}_{r,\mathcal{B}}(t) = Q^{rec}_{r,\mathcal{B}}(t) - Q^{rem}_{r}(t), \tag{44}$$

$$\Delta^{rec}_{j,r,\mathcal{B}}(t) = Q^{rec}_{r,\mathcal{B}}(t) - Q^{rec,-j}_{r,\mathcal{B}}(t). \tag{45}$$

If $Q^{rec}_{r,\mathcal{B}}(t) \geq Q^{rem}_{r}(t)$, then j is a mandatory rescue job if and only if

$$\Delta^{rec}_{j,r,\mathcal{B}}(t) > S^{rec}_{r,\mathcal{B}}(t). \tag{46}$$

PROOF. By Definition 2, j is mandatory if and only if

$$Q^{rec,-j}_{r,\mathcal{B}}(t) < Q^{rem}_{r}(t).$$

Substituting

$$Q^{rec,-j}_{r,\mathcal{B}}(t) = Q^{rec}_{r,\mathcal{B}}(t) - \Delta^{rec}_{j,r,\mathcal{B}}(t)$$

gives

$$Q^{rec}_{r,\mathcal{B}}(t) - \Delta^{rec}_{j,r,\mathcal{B}}(t) < Q^{rem}_{r}(t),$$

which is equivalent to

$$\Delta^{rec}_{j,r,\mathcal{B}}(t) > Q^{rec}_{r,\mathcal{B}}(t) - Q^{rem}_{r}(t) = S^{rec}_{r,\mathcal{B}}(t).$$

Implication for NR-RG-RHO-LNS: when $S^{rec}_{r,\mathcal{B}}(t)$ is thin, even jobs with moderate marginal contributions become rescue-critical, providing a direct signal for fragile entity identification and service-risk-aware diversity.

4.5. Implications for algorithm design

The structural results do not imply that every RG move improves the final schedule. Instead, they provide service-risk-aware information that can be used to diversify candidates and neighborhoods. NR-RG-RHO-LNS uses these indicators in five ways:

1. Recoverability diagnosis: $Q^{rec}_{r,\mathcal{B}}(t)$, $W^{\min}_{r,\mathcal{B}}(t)$, and $U_{r,\mathcal{B}}(t)$ quantify residual service risk at each event.
2. Fragile entity identification: $S^{rec}_{r,\mathcal{B}}(t)$ separates entities with a recovery margin from those close to losing recoverability.
3. Mandatory rescue candidate generation: Definition 2 and Propositions 1–3 identify jobs that should appear in RG-oriented construction and repair candidates.
4. No-regret filtering: all RG candidates are evaluated by Z; inferior candidates are not forced into the incumbent.
5. Service-risk-aware diversity: RG-SPT and RG-ATC hybrid candidates complement generic earliest-deadline and processing-time candidates.

Algorithm 1 Overall NR-RG-RHO-LNS framework

Require: SL-ISP instance, event stream, schedule-evaluation budget
Ensure: Best complete schedule evaluated by Z

```
Generate initial incumbents by Algorithm 2
Set the best incumbent to the schedule with the smallest Z
for each event-driven decision epoch do
    Update job releases, completed operations, idle machines, and ready operations
    Diagnose recoverability using Q^rec_{r,B}(t), U_{r,B}(t), and mandatory rescue jobs
    Build a rolling-horizon candidate set from ready jobs, deadline-risk jobs, and service-risk jobs
    for each retained incumbent do
        Generate generic LNS candidates using unbiased destroy and repair moves
        Generate RG candidates using fragile entities and mandatory rescue jobs
        Evaluate every candidate schedule by the original objective Z
        Accept an improving candidate; otherwise preserve the incumbent
        Update the best incumbent if a smaller Z is found
    end for
end for
return the best incumbent according to Z
```

Algorithm 2 No-regret multi-start initialization

Require: Initial schedule state and construction budget
Ensure: Retained incumbent pool

```
Initialize an empty candidate pool
Add EDF and EDD-oriented construction candidates
Add SPT, WSPT, ATC, and regret-k construction candidates
Diagnose initial recoverability and compute service-risk priorities
Add RG-SPT and RG-ATC hybrid candidates
for each candidate schedule do
    Evaluate the candidate by Z
    Apply lightweight polishing under the same feasibility rules
    Re-evaluate the polished candidate by Z
end for
Sort all candidates by Z
Retain the top-ranked schedules as the initial incumbent pool
return retained incumbents
```

5. Proposed algorithm: NR-RG-RHO-LNS

5.1. Overall framework

NR-RG-RHO-LNS is a solver-free algorithm for event-driven SL-ISP. It combines a rolling-horizon optimization backbone with large neighborhood search, but does not solve MILP or CP subproblems during the main search. At each event epoch, the algorithm updates the available operations, maintains a pool of incumbent schedules, diagnoses recoverability, generates generic and RG candidates, evaluates all candidates by Z, and preserves the best incumbent found so far.

5.2. No-regret multi-start initialization

The initialization stage constructs a diverse candidate pool. Candidate schedules are generated from earliest deadline first (EDF), earliest due date (EDD)-oriented priority, SPT, WSPT, ATC, regret-k insertion, RG-SPT hybrids, and RG-ATC hybrids. Each candidate is evaluated by Z, lightly polished by local improving moves, and re-evaluated. The top-ranked schedules are retained as initial incumbents. The no-regret rule means that recoverability-guided candidates are allowed to enter the pool, but the final initial incumbent is selected only by Z.

5.3. Recoverability-guided candidate generation

RG candidate generation uses the structural quantities in Section 4. Entities with positive $U_{r,\mathcal{B}}(t)$ are recognized as partially unrecoverable under the optimistic relaxation; entities with small $S^{rec}_{r,\mathcal{B}}(t)$ are treated as fragile. Mandatory rescue jobs are inserted into candidate priority lists, and jobs with high marginal contribution $\Delta^{rec}_{j,r,\mathcal{B}}(t)$ are used to diversify RG-oriented schedules.

The RG component is guarded. It can propose construction and repair candidates, but it does not override the objective. A candidate that improves a recoverability indicator but worsens Z is retained only if it survives the same acceptance rule applied to all candidates. This design prevents recoverability guidance from becoming a hard constraint that could degrade the incumbent.

5.4. Rolling-horizon LNS search

The RHO-LNS stage operates on a restricted set of jobs around the current event epoch. Completed operations and ongoing non-preemptive operations are fixed. The search destroys a subset of scheduled decisions in the rolling horizon and repairs them using feasible dispatching and insertion rules. Generic neighborhoods target deadline-risk jobs, congested machine intervals, and high-tardiness jobs. RG neighborhoods target fragile entities, mandatory rescue jobs, and operations competing for the bottleneck pool $\mathcal{B}$.

All repaired schedules are evaluated by Z. The search budget is counted by schedule evaluations so that iterative methods can be compared under an equal computational budget in Section 6.

5.5. Incumbent preservation and acceptance rule

The incumbent preservation rule is simple: the best schedule found so far is never replaced by a schedule with worse Z. During local search, a candidate is accepted if it strictly improves Z. A guarded acceptance option may allow a candidate with the same Z to be kept when it improves WSF or increases service-risk diversity, but final selection is always based on the original objective Z. This rule is the "no-regret" part of NR-RG-RHO-LNS.

5.6. Computational complexity and implementation notes

The computational effort of NR-RG-RHO-LNS is controlled by the number of retained incumbents, the rolling-horizon candidate size, the number of LNS iterations, and the cost of recoverability diagnosis. The recoverability relaxation decomposes by service entity and can be solved by dynamic programming for the 0-1 knapsack form in (32)–(33). The main search remains solver-free: it uses constructive scheduling, destroy-repair moves, local polishing, and objective evaluation rather than repeatedly invoking the MILP.

6. Computational experiments

6.1. Experimental design

The experiments are designed to evaluate algorithmic behavior under controlled service pressure, machine capacity, and job-size settings. The planned benchmark uses three pressure levels, four job-size levels, three machine-size levels, and matched random seeds. The current experimental configuration is summarized in Table 3. These values describe the planned design and do not constitute performance results.

Matched seeds are used so that paired comparisons are made on the same instance and stochastic stream. Iterative methods are allocated an equal schedule-evaluation budget. If objective-weight or entity-weight sensitivity files are generated, they will be reported as supplementary analyses rather than used to replace the main comparison.

6.2. Compared algorithms

Table 4 lists the 13 algorithms planned for the main comparison. The table describes mechanisms and solver use; it does not report performance.

6.3. Evaluation metrics

Each run will report the final objective Z, total tardiness $TT = \sum_{j\in\mathcal{J}} T_j$, weighted service shortfall $WSF = \sum_{r\in\mathcal{R}} w_r U_r$, zero-shortfall entity rate $ZSR = |\{r : U_r = 0\}|/|\mathcal{R}|$, runtime, and schedule-evaluation counts where available. The same objective weights used in scheduling are used for final evaluation.

Table 3
Experimental settings. Planned instance factors and run structure.

Item	Planned setting
Instance factors	3 pressure levels × 3 machine levels × 4 job-size levels
Jobs	40, 80, 120, 160
Machines	5, 10, 15
Service entities	Scaled with job count; planned levels include 3, 5, 8, and 10
Operations per job	Uniformly generated in the range 2–4
Processing times	Generated from the configured interval [10, 50]
Transportation delays	Generated from the configured interval [10, 40]
Pressure levels	Medium, high, and very high fulfillment-ratio settings
Dynamic arrivals	Medium-intensity dynamic arrivals with matched seeds
Objective weights	$\alpha = 1.0$, $\beta = 1.0$ in the main design
Compared algorithms	13 algorithms listed in Table 4
Run count	Planned total: 1404 runs under the current configuration

Table 4
Algorithm classification.

Algorithm	Category	Main mechanism	Solver-free
EDD	Dispatching rule	Earliest due date priority based on entity cutoff.	Yes
SPT	Dispatching rule	Shortest processing time priority.	Yes
WSPT	Dispatching rule	Processing-time priority weighted by service importance.	Yes
ATC	Dispatching rule	Apparent tardiness cost priority with deadline slack.	Yes
Service-weighted	Service rule	Priority biased toward high-weight entities.	Yes
Shortfall-greedy	Service rule	Greedy priority based on immediate shortfall reduction.	Yes
Plain RHO	Rolling horizon	EDF-style construction in a rolling horizon without LNS.	Yes
RHO-LNS	LNS baseline	Rolling-horizon LNS without recoverability guidance.	Yes
Adaptive RG	RG baseline	Recoverability-guided LNS without the final no-regret multi-start design.	Yes
ILS	Metaheuristic	Iterated local search adapted to the event-driven setting.	Yes
VNS	Metaheuristic	Variable neighborhood search with rolling-horizon evaluation.	Yes
TS	Metaheuristic	Tabu search with local swap or relocation neighborhoods.	Yes
NR-RG-RHO-LNS	Proposed method	No-regret multi-start, RG candidate diversity, RHO-LNS, and incumbent preservation.	Yes

6.4. Overall performance comparison

Table 5 reports the overall performance of all compared methods. The final interpretation will be completed after the experimental results are generated.

6.5. Pressure-level analysis

Table 6 will compare all algorithms under the three pressure levels. Figures 3 and 4 will visualize the corresponding Z and WSF patterns. The analysis will be written only after the pressure-level aggregates are available.

Table 5
Overall comparison. Cells will be populated from the final run-level CSV.

Algorithm	Z	TT	WSF	ZSR	Runtime
EDD	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
SPT	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
WSPT	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
ATC	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Service-weighted	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Shortfall-greedy	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Plain RHO	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
RHO-LNS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Adaptive RG	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
ILS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
VNS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
TS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
NR-RG-RHO-LNS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*

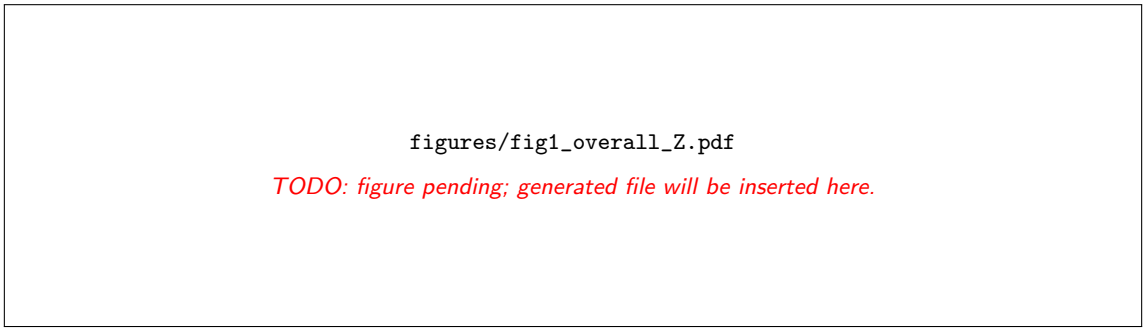

Figure 1: Overall Z comparison across the 13 algorithms. Error bars or distribution summaries will be inserted after aggregation.

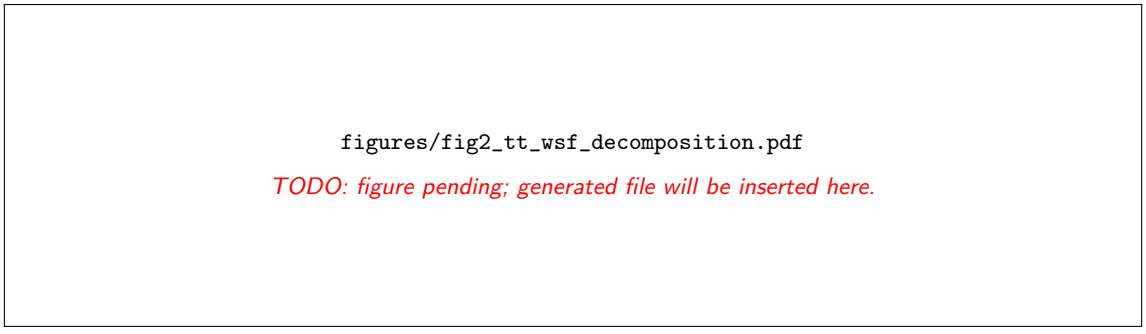

Figure 2: Decomposition of the objective into TT and WSF components.

6.6. Statistical tests

Paired tests will compare NR-RG-RHO-LNS with each baseline using matched instance-seed pairs. The primary test is a paired nonparametric test on Z, with win rate and effect size reported together. Run-level and instance-level summaries will both be retained to avoid over-interpreting repeated seeds as independent instances.

6.7. Ablation study

The ablation study will isolate the contribution of construction diversity, no-regret selection, lightweight polishing, top-ranked incumbent selection, and RG-LNS search. The planned variants are listed in Table 9.

Table 6
Pressure-level comparison. The final table contains 3 pressure levels × 13 algorithms.

Pressure	Algorithm	Z	TT	WSF	ZSR
Medium / High / Very high	13 algorithms	*TODO: pending aggregation from run-level CSV*			

figures/fig3_pressure_Z.pdf

TODO: figure pending; generated file will be inserted here.

Figure 3: Pressure-level Z comparison for representative and full algorithm groups.

figures/fig4_pressure_WSF.pdf

TODO: figure pending; generated file will be inserted here.

Figure 4: Pressure-level WSF comparison, highlighting service-shortfall behavior under increasing fulfillment requirements.

Table 7
Paired statistical tests. NR-RG-RHO-LNS will be compared with the 12 baselines.

Baseline	ΔZ	Win rate	p-value	Effect size
12 baselines	*TODO: pending paired tests from matched run-level results*			

6.8. Scalability analysis

Scalability will be evaluated by job size and machine size. Table 10 reserves columns for relative gap to the best observed method and improvement over RHO-LNS, rather than reporting raw Z alone.

6.9. Convergence and runtime analysis

Convergence curves are required for the iterative methods. The convergence figure will report best-so-far Z versus iteration under the matched evaluation budget. Runtime will be shown on a log scale where appropriate. Table 11 reserves budget-related fields, including schedule evaluations, construction time, and search time.

6.10. Operator contribution

Operator diagnostics will be reported only when operator-level logs are available. Table 12 and Figure 10 reserve space for usage ratio, success rate, and mean improvement magnitude.

Table 8
Pressure-specific paired tests. This table is reserved for appendix or supplementary reporting.

Pressure	Baseline	ΔZ	Win rate	p-value
Medium / High / Very high	12 baselines	*TODO: pending pressure-specific paired tests*		

Table 9
Ablation study. Values will be computed from ablation or tagged run-level outputs.

Variant	Z	TT	WSF	ZSR
EDF-only	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Best-of-5	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Multi-start	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
+polishing	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
+top-k LNS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*
Full NR-RG-RHO-LNS	*TODO: pending*	*TODO: pending*	*TODO: pending*	*TODO: pending*

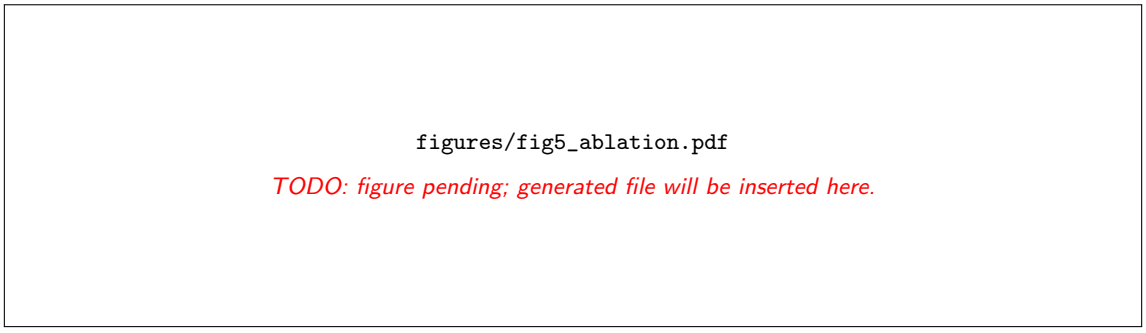

Figure 5: Ablation study comparing incremental components of NR-RG-RHO-LNS.

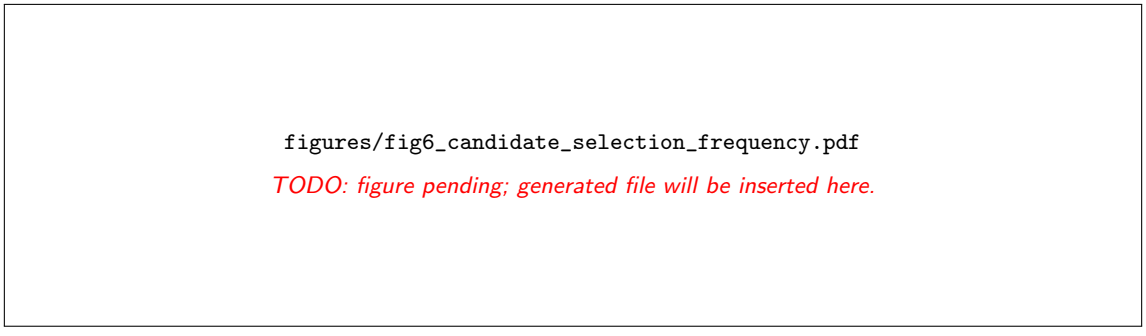

Figure 6: Candidate selection frequency during no-regret multi-start initialization.

6.11. Gantt-chart illustration

A representative Gantt chart is required to illustrate the interaction between machine schedules and service-entity fulfillment. The final example will be selected after the runs are complete and will be described without overstating generality.

7. Conclusions

This manuscript framework defines SL-ISP, presents a compact offline MILP reference model, develops structural recoverability indicators, and specifies the solver-free NR-RG-RHO-LNS algorithm. The recoverability analysis

Table 10
Scalability analysis. Relative gap and improvement columns prevent the analysis from relying only on raw Z.

Jobs	Machines	Algorithm	Gap to best	Improvement over RHO-LNS
40 / 80 / 120 / 160	5 / 10 / 15	13 algorithms	*TODO: pending scalability aggregation*	

Figure 7: Scalability heatmap based on relative gap or improvement over RHO-LNS.

Table 11
Runtime and budget.

Algorithm	Avg. runtime	Max. runtime	Evaluations	Construction time	Search time
13 algorithms	*TODO: pending runtime and budget logs*				

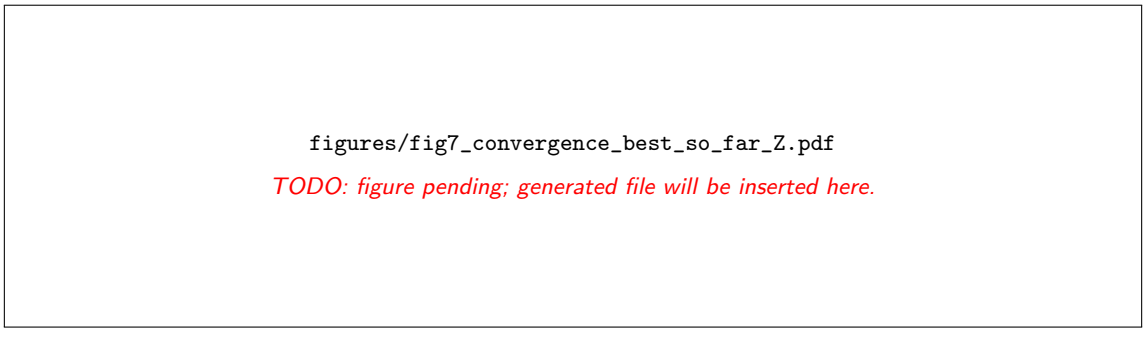

Figure 8: Convergence curve showing best-so-far Z versus iteration for iterative methods.

provides diagnostic quantities for fragile entities, unavoidable shortfall, and mandatory rescue jobs, while the algorithm uses these quantities for candidate diversity under no-regret incumbent preservation.

The empirical conclusions are not written in this version because the final experimental results are not yet available. After the result CSV files and figures are generated, this section should be updated with verified findings on overall performance, pressure sensitivity, statistical significance, ablation effects, scalability, convergence, runtime, and the representative Gantt-chart case.

References

[1] Arora, D., Agarwal, G., 2016. Meta-heuristic approaches for flowshop scheduling problems: a review. International Journal of Advanced Operations Management 8, 1–16.

[2] Burke, E.K., Landa Silva, J., 2005. The design of memetic algorithms for scheduling and timetabling problems, in: Recent advances in memetic algorithms. Springer, pp. 289–311.

[3] Hou, Y., Fu, Y., Gao, K., Zhang, H., Sadollah, A., 2022. Modelling and optimization of integrated distributed flow shop scheduling and distribution problems with time windows. Expert Systems with Applications 187, 115827. URL: https://linkinghub.elsevier.com/

figures/fig8_runtime_log_scale.pdf

TODO: figure pending; generated file will be inserted here.

Figure 9: Runtime comparison across algorithms on a log scale.

Table 12
Operator contribution.

Operator group	Usage ratio	Success rate	Mean ΔZ improvement
Destroy / repair / RG-hybrid operators	*TODO: pending operator diagnostics*		

figures/fig10_operator_contribution.pdf

TODO: figure pending; generated file will be inserted here.

Figure 10: Operator contribution measured by success rate and improvement magnitude.

figures/fig11_representative_gantt.pdf

TODO: figure pending; generated file will be inserted here.

Figure 11: Representative Gantt chart showing machine assignments and service-entity fulfillment timing.

retrieve/pii/S0957417421011921, doi:10.1016/j.eswa.2021.115827.

[4] Hou, Y., Guo, X., Han, H., Wang, J., 2023. Knowledge-driven ant colony optimization algorithm for vehicle routing problem in instant delivery peak period. Applied Soft Computing 145, 110551. URL: https://linkinghub.elsevier.com/retrieve/pii/S1568494623005690, doi:10.1016/j.asoc.2023.110551.

[5] Huang, J.P., Gao, L., Li, X.Y., 2024. A hierarchical multi-action deep reinforcement learning method for dynamic distributed job-shop scheduling problem with job arrivals. IEEE Transactions on Automation Science and Engineering 22, 2501–2513.

[6] Jia, Y., Wang, W., Zhang, J., 2025. A learning-and-knowledge-assisted multi-population collaborative evolutionary algorithm for integrated design-production-distribution scheduling problems in mass personalized customization. Swarm and Evolutionary Computation 99, 102158. URL: https://linkinghub.elsevier.com/retrieve/pii/S2210650225003153, doi:10.1016/j.swevo.2025.102158.

[7] Liu, C.L., Tseng, C.J., Huang, T.H., Wang, J.W., 2023. Dynamic parallel machine scheduling with deep q-network. IEEE Transactions on Systems, Man, and Cybernetics: Systems 53, 6792–6804.

[8] Sawik, T., 2016. Integrated supply, production and distribution scheduling under disruption risks. Omega 62, 131–144. URL: https://linkinghub.elsevier.com/retrieve/pii/S030504831500198X, doi:10.1016/j.omega.2015.09.005.

[9] Yuan, E., Wang, L., Cheng, S., Song, S., Fan, W., Li, Y., 2024. Solving flexible job shop scheduling problems via deep reinforcement learning. Expert Systems with Applications 245, 123019. URL: https://linkinghub.elsevier.com/retrieve/pii/S0957417423035212, doi:10.1016/j.eswa.2023.123019.

[10] Zhang, Z., Fu, Y., Gao, K., Zhang, H., Wang, L., 2024. A cooperative evolutionary algorithm with simulated annealing for integrated scheduling of distributed flexible job shops and distribution. Swarm and Evolutionary Computation 85, 101467. URL: https://linkinghub.elsevier.com/retrieve/pii/S2210650223002390, doi:10.1016/j.swevo.2023.101467.

[11] Zhao, S., Zhou, H., 2025. Learning-driven memetic algorithm for solving integrated distributed production and transportation scheduling problem. Swarm and Evolutionary Computation 96, 101945. URL: https://linkinghub.elsevier.com/retrieve/pii/S2210650225001038, doi:10.1016/j.swevo.2025.101945.